



**UNIVERSIDAD
TECNOLÓGICA
DE PUEBLA**

**UNIVERSIDAD TECNOLÓGICA DE PUEBLA
INGENIERÍA EN DESARROLLO DE
SOFTWARE MULTIPLATAFORMA**

**APLICACIÓN
WEB**

REPORTE — SUBPRODUCTO NO. 2

**ASIGNATURA: APLICACIONES WEB
PROGRESIVAS**

**DOCENTE: MATHER XÓCHITL MENDOZA
PÍSCIL**

**ALUMNO: PÉREZ ALCÁNTARA ALEJANDRO
10° “D”**

FECHA: 9 DE SEPTIEMBRE DE 2026

ÍNDICE

1. Introducción	3
2. ¿Qué es una Aplicación Web?	3
2.1 Tipos de Aplicaciones Web	3
3. Ilustración: ¿Cómo funciona una Aplicación Web?	4
4. Características Principales	5
5. Ventajas y Desventajas	5
6. Comparativa: Web vs. Nativa vs. Híbrida	6
7. Tecnología Utilizada	7
8. Ejemplo de Funcionamiento: MediCore Professional System	7
9. Tabla Comparativa: Requisitos de una Aplicación Web vs. MediCore	14
10. Lista de Cotejo	16
11. Conclusión	17
12. URL de la Aplicación	17
Referencias	18

1. Introducción

En el desarrollo de software existen distintos enfoques para construir aplicaciones: nativo, web e híbrido. Ya documenté el enfoque nativo en el Subproducto No. 1 de esta materia, usando como ejemplo propio mi proyecto de Wear OS. En este segundo reporte me enfoco en el enfoque contrario: las aplicaciones web, el tipo de desarrollo que permite que un mismo programa funcione desde cualquier dispositivo con navegador, sin necesidad de instalación ni de mantener versiones separadas por sistema operativo.

A lo largo del documento defino el concepto de aplicación web, ilustro su funcionamiento mediante un diagrama, listo sus características principales, sus ventajas y desventajas, la comparo frente a los enfoques nativo e híbrido, documento la tecnología que se usa para construirla y, como evidencia propia, presento mi propio proyecto: MediCore Professional System, una plataforma clínica web que desarrollé y que actualmente tengo desplegada en la nube. Cierro el documento con una tabla comparativa entre lo que exige la definición de una aplicación web y lo que realmente implementé en MediCore, y con una lista de cotejo de autoevaluación del propio reporte.

2. ¿Qué es una Aplicación Web?

Una aplicación web es un programa de software al que se accede a través de un navegador de internet (Chrome, Edge, Firefox, Safari), sin necesidad de instalarlo en el dispositivo. A diferencia de una aplicación nativa —que se compila específicamente para iOS o Android y se instala desde una tienda de aplicaciones—, una aplicación web se ejecuta del lado del servidor y del navegador: el servidor procesa la lógica de negocio y los datos, y el navegador se encarga de mostrar la interfaz y de la interacción con el usuario.

Esto permite que la misma aplicación funcione igual en una computadora, una tablet o un teléfono, sin necesidad de crear versiones distintas para cada sistema operativo, y que cualquier actualización se refleje de inmediato para todos los usuarios, sin que ellos tengan que descargar nada. Es exactamente el mismo principio que expliqué en el Subproducto No. 1, mirado desde el lado opuesto: ahí documenté cómo el desarrollo nativo depende de un código independiente por plataforma; aquí documento cómo el desarrollo web resuelve ese mismo problema con un solo código fuente que corre dentro del navegador.

2.1 Tipos de Aplicaciones Web

No todas las aplicaciones web funcionan igual internamente. Distingo cuatro tipos principales, que fui identificando conforme investigué para este reporte:

- **Sitios web estáticos:** el contenido es fijo y se envía tal cual desde el servidor; no cambia según el usuario ni permite interacción real (por ejemplo, una página informativa simple).

- **Aplicaciones web dinámicas:** el servidor genera el contenido en tiempo real según la solicitud, normalmente consultando una base de datos. MediCore entra en esta categoría: cada vista se arma con PHP consultando MySQL.
- **Single Page Applications (SPA):** cargan una sola vez y luego actualizan el contenido sin recargar la página completa, comunicándose con el servidor en segundo plano (por ejemplo, Gmail o Trello).
- **Progressive Web Apps (PWA):** aplicaciones web que además pueden instalarse en el dispositivo, funcionar parcialmente sin conexión y enviar notificaciones push, acercándose a la experiencia de una app nativa sin dejar de ser, en el fondo, una aplicación web.

3. Ilustración: ¿Cómo funciona una Aplicación Web?

El siguiente diagrama ilustra el flujo general de una aplicación web: desde que el usuario escribe la URL en su navegador, hasta que el servidor procesa la petición, consulta la base de datos si es necesario, y regresa una respuesta que el navegador renderiza en pantalla.

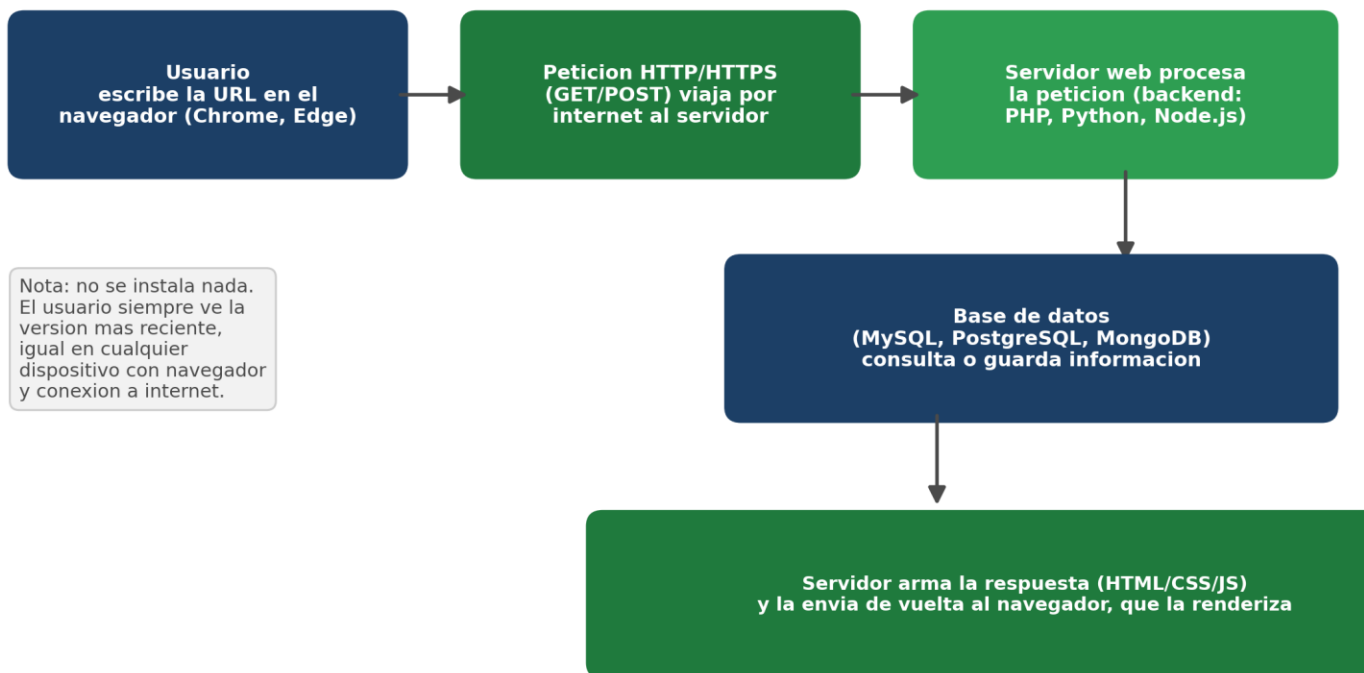


Figura 1. Flujo de funcionamiento de una aplicación web, desde el navegador hasta el servidor y la base de datos.

4. Características Principales

- **No requiere instalación:** se accede directamente desde el navegador mediante una URL, sin descargar ni instalar nada en el dispositivo.
- **Multiplataforma por naturaleza:** el mismo código funciona igual en Windows, macOS, Linux, Android o iOS, ya que todo se ejecuta dentro del navegador.
- **Actualización centralizada:** al actualizar el servidor, todos los usuarios ven la versión más reciente de inmediato, sin necesidad de descargar parches.
- **Depende de conexión a internet:** la mayoría de sus funciones requieren comunicación constante con el servidor (salvo en aplicaciones web progresivas con soporte offline limitado).
- **Acceso limitado al hardware:** a diferencia de una app nativa, solo puede usar las funciones del dispositivo que el navegador expone mediante APIs web (cámara, ubicación, notificaciones).
- **Seguridad basada en el servidor:** la protección de datos, la autenticación y el control de acceso se gestionan del lado del servidor (sesiones, tokens, cifrado de contraseñas).

5. Ventajas y Desventajas

A partir de las características anteriores, resumo las principales ventajas y desventajas del desarrollo web frente a otros enfoques, tal como hice en el Subproducto No. 1 para el desarrollo nativo:

VENTAJAS	DESVENTAJAS
No requiere descarga ni instalación: se accede de inmediato por URL.	Depende casi por completo de tener conexión a internet.
Un solo código fuente funciona en cualquier sistema operativo.	Acceso limitado al hardware del dispositivo comparado con una app nativa.
Actualizaciones instantáneas para todos los usuarios, sin parches.	El rendimiento suele ser menor que el de una aplicación nativa equivalente.
Menor costo y tiempo de desarrollo: no hay que mantener varias versiones por plataforma.	La experiencia de usuario depende del navegador y puede variar entre ellos.
Más fácil de mantener, dar soporte y corregir errores de forma centralizada.	Mayor exposición a ataques web (XSS, CSRF, inyección SQL) si no se programa con cuidado.

6. Comparativa: Web vs. Nativa vs. Híbrida

Para dimensionar mejor a la aplicación web dentro del panorama general del desarrollo de aplicaciones, la comparo frente a los otros dos enfoques que ya documenté en el Subproducto No. 1: el enfoque nativo y el enfoque híbrido.

CRITERIO	WEB	NATIVA	HÍBRIDA
Lenguaje	HTML, CSS, JavaScript (+ backend: PHP, etc.)	Swift/Kotlin (por plataforma)	HTML/JS + wrapper (Flutter, React Native, Ionic)
Rendimiento	Medio-bajo	Alto	Medio
Acceso al hardware	Limitado (APIs del navegador)	Completo	Amplio, vía plugins/puentes nativos
Instalación	No requiere, se accede por URL	Tienda oficial (App Store/Google Play)	Tienda oficial
Funciona sin internet	Muy limitado	Sí	Sí, parcialmente
Costo de desarrollo	Bajo (1 solo código)	Alto (2+ códigos independientes)	Medio (1 código compartido)
Ejemplos	MediCore, Google Docs, PWA	Spotify, mi app Wear OS	Uber Eats, versiones iniciales de Instagram

La aplicación web es la opción con menor costo y tiempo de desarrollo, y la más sencilla de mantener, pero también la de menor rendimiento y menor acceso al hardware frente al desarrollo nativo. Fue justo ese equilibrio —bajo costo, mantenimiento centralizado, acceso multiplataforma— lo que me hizo elegir el enfoque web para MediCore, un sistema pensado para que distintos médicos lo usen desde distintos dispositivos sin que yo tenga que mantener una app por cada sistema operativo.

7. Tecnología Utilizada

El desarrollo web se divide en dos partes que trabajan en conjunto: el frontend (lo que ve y usa el usuario) y el backend (la lógica y los datos, del lado del servidor). En MediCore utilicé la siguiente tecnología para cada capa:

COMPONENTE	TECNOLOGÍA	FUNCIÓN
Frontend	HTML5, CSS3, JavaScript	Estructura, diseño visual e interactividad en el navegador
Backend	PHP	Lógica de negocio, autenticación, sesiones y procesamiento de datos
Base de datos	MySQL	Almacenamiento de la información (pacientes, expedientes, citas)
Servidor / Hosting	Docker + Railway (nube)	Ejecuta la aplicación y la expone públicamente mediante una URL
Seguridad	Bcrypt, CSRF Tokens, RBAC	Cifrado de contraseñas, protección de formularios y control de acceso por rol

Elegí PHP y MySQL porque es la pila con la que llevo trabajando MediCore desde el inicio del cuatrimestre, y porque es la combinación clásica para este tipo de sistema de gestión con formularios, tablas relacionales y generación de reportes. Para la seguridad, en vez de reinventar nada, me apoyé en prácticas ya establecidas de la industria (Bcrypt para contraseñas, tokens CSRF en cada formulario, sesiones controladas del lado del servidor) que documenté a detalle en trabajos anteriores de esta misma materia.

8. Ejemplo de Funcionamiento: MediCore Professional System

Como ejemplo real de aplicación web, presento mi propio proyecto: MediCore Professional System, una plataforma clínica que desarrollé para centralizar la gestión de pacientes, expedientes médicos, citas y reportes de una institución de salud. MediCore está construida con PHP y MySQL en el backend, HTML/CSS/JavaScript en el frontend, y actualmente se encuentra desplegada en la nube (Railway) mediante un contenedor Docker, accesible públicamente desde cualquier navegador sin necesidad de instalar nada.

A continuación documento con capturas reales el funcionamiento del sistema completo, desde el portal público y el acceso, hasta el panel interno de gestión clínica: dashboard, pacientes, expedientes, agenda, reportes, recomendaciones automáticas, material educativo y configuración de perfil.



Figura 2. Portal público (home.php): página de bienvenida de MediCore Professional System.

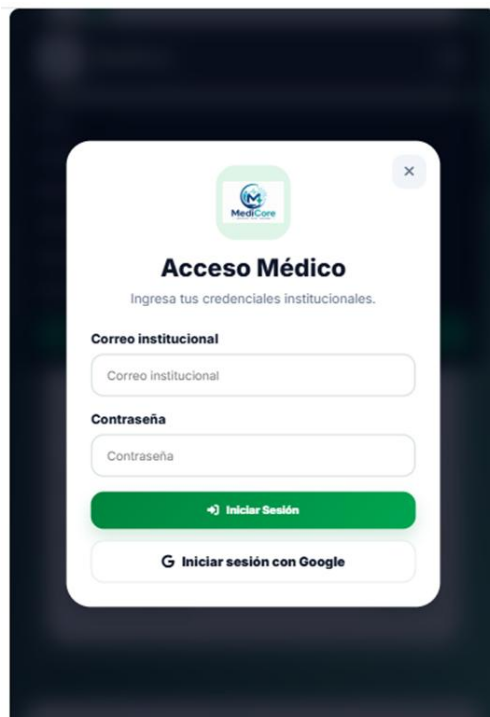


Figura 3. Modal de Acceso Médico: formulario de inicio de sesión con credenciales institucionales.



Figura 4. Dashboard principal: panel de bienvenida con estadísticas clínicas generales.

Pacientes Registrados

PACIENTE	CORREO	TELÉFONO	ESTADO
Alejandro Pérez Alcántara	doctor@nutrvida.com	223228151	Autorizada
Antonio Pérez	antonio@gmail.com	No registrado	Autorizada
Aurora Evangeline	aurora@gmail.com	No registrado	Autorizada
Charles Leclerc	charles@gmail.com	No registrado	Pendiente
Daniel Duran Duran	Daniel@duran.com	No registrado	Autorizada
Franco Lopez	fran4.lopez@gmail.com	2223679823	Autorizada
Ilia Tupuria	tupuria09@gmail.com	No registrado	Autorizada
Laura Sánchez	laura@gmail.com	No registrado	Autorizada
Lewis	lewis@gmail.com	No	Pendiente

Dashboard Pacientes Agenda Reportes Recomendaciones Enfermedades Ubicación

Figura 5. Módulo de Pacientes: listado de pacientes registrados con estado de autorización.

Plan de Entrenamiento

Cardio como natación y pesas

Estatus del Plan

Autorizada

Seguimiento Médico

Fecha de Cita

10/06/2026

Hora de Cita

10:25 a. m.

Consulta por videollamada

Se generó una sala de videollamada para la cita programada. Compartela con el paciente y usa este mismo botón para entrar tú también.

Iniciar videollamada

Guardar Expediente

Dashboard
Pacientes
Agenda
Reportes
Recomendaciones
Enfermedades
Utilidad

Figura 6. Expediente Clínico: plan de entrenamiento, seguimiento médico y videollamada de consulta.

Agenda Clínica Profesional

Administra consultas médicas, seguimientos nutricionales y citas programadas dentro del ecosistema profesional MediCore.

Pacientes
Dashboard

0
CITAS PROGRAMADAS

0
CONSULTAS HOY

0
PENDIENTES

0
AUTORIZADAS

Dashboard
Pacientes
Agenda
Reportes
Recomendaciones
Enfermedades
Utilidad

Figura 7. Agenda Clínica Profesional: administración de consultas y citas programadas.

Últimos Pacientes Registrados		
PACIENTE	CORREO	FECHA REGISTRO
Alejandro Pérez Alcántara	doctor@nutrvida.com	13/04/2026 19:50
maria ramirez martinez	maria@g.com	13/04/2026 19:42
Stephen Curry	curry@cheft.com	10/04/2026 09:08
Franco Lopez	fran4.lopez@gmail.com	25/03/2026 07:18
Daniel Duran Duran	Daniel@duran.com	19/03/2026 21:22

Figura 8. Módulo de Reportes: últimos pacientes registrados en el sistema.

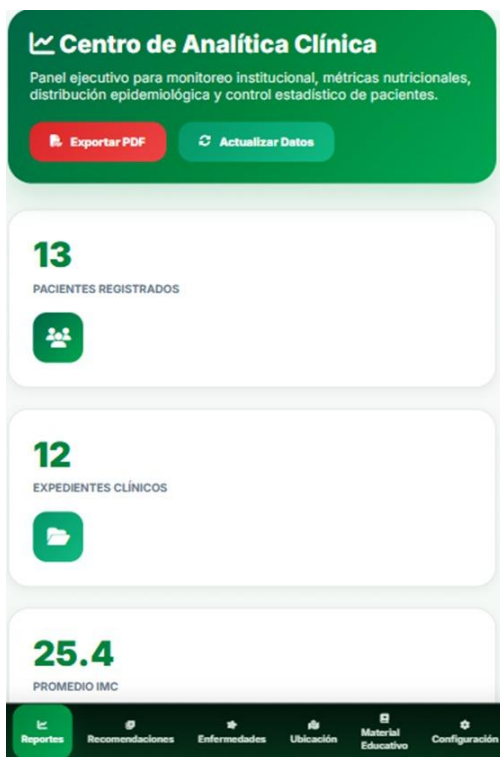


Figura 9. Centro de Analítica Clínica: panel ejecutivo con métricas institucionales.

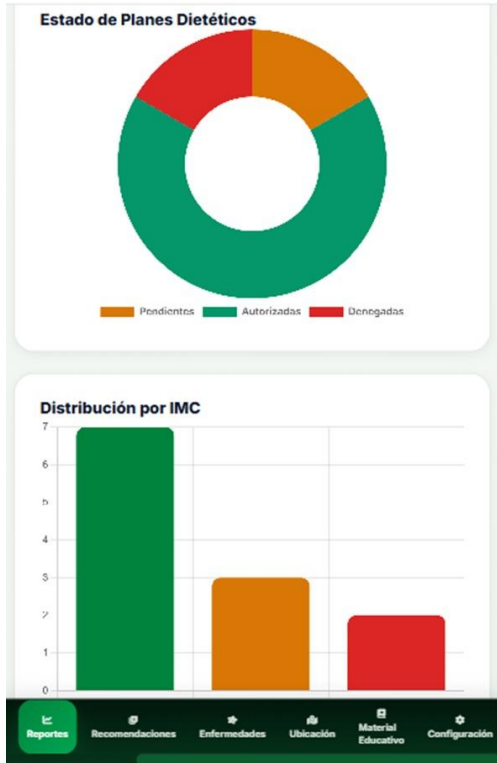


Figura 10. Gráficas estadísticas (Chart.js): estado de planes dietéticos y distribución por IMC.

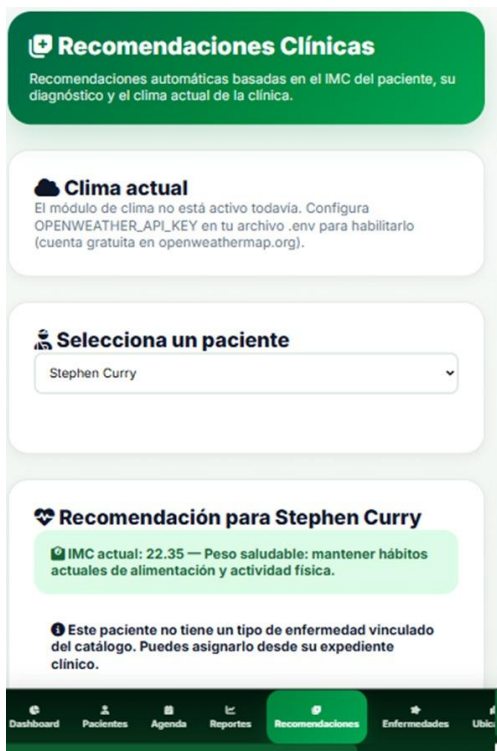


Figura 11. Recomendaciones Clínicas automáticas basadas en el IMC del paciente.



Figura 12. Biblioteca Educativa: infografía clínica de apoyo para pacientes.



Figura 13. Biblioteca Educativa: objetivo educativo y recomendaciones de prevención.

Haydeé Gómez Díaz

Correo Electrónico

haydee.gomez@utpuebla.edu.mx

Guardar Cambios

Seguridad y Contraseña

Recomendaciones de seguridad:

- Utiliza mínimo 8 caracteres.
- Incluye letras mayúsculas y números.
- No compartas tus credenciales.
- Actualiza tu contraseña regularmente.

Contraseña Actual

Ingresa tu contraseña actual

Nueva Contraseña

Nueva contraseña segura

Confirmar Nueva Contraseña

Repite la nueva contraseña

Actualizar Contraseña

Dashboard Pacientes Agenda Reportes Recomendaciones Enfermedades Ubicación

Figura 14. Configuración Profesional: seguridad y actualización de contraseña.

9. Tabla Comparativa: Requisitos de una Aplicación Web vs. MediCore

De la misma forma en que en el Subproducto No. 1 comparé mi proyecto de Wear OS contra lo que exige la definición de una aplicación nativa, aquí reviso, criterio por criterio, si MediCore cumple con lo que se espera de una aplicación web real:

CRITERIO DE APP WEB	QUÉ EXIGE LA DEFINICIÓN	¿CUMPLE?	EVIDENCIA EN MEDICORE
Sin instalación	Debe funcionar solo con abrir la URL en el navegador	Sí	Se accede directo en home.php, sin descargar nada
Backend en servidor	La lógica de negocio debe procesarse del lado del servidor	Sí	PHP procesa login, expedientes, citas y reportes en Railway
Base de datos centralizada	Los datos deben persistir en un servidor, no en el dispositivo	Sí	MySQL en la nube almacena pacientes, expedientes y citas
Multiplataforma real	Debe verse y funcionar igual en cualquier sistema operativo	Sí	Probado en escritorio y en vista móvil (responsive) desde el mismo código
Acceso mediante URL pública	Debe estar disponible en internet con una dirección accesible	Sí	https://practicaequipogithub-production.up.railway.app

CRITERIO DE APP WEB	QUÉ EXIGE LA DEFINICIÓN	¿CUMPLE?	EVIDENCIA EN MEDICORE
Autenticación segura	Debe proteger el acceso con credenciales y control de sesión	Sí	Login con Bcrypt, sesiones PHP y segundo factor opcional (Twilio OTP)
Protección de formularios	Debe prevenir ataques comunes (CSRF, XSS, SQL Injection)	Sí	Tokens CSRF, prepared statements y sanitización de entradas
Control de acceso por rol	Debe restringir funciones según el tipo de usuario	Sí	RBAC: separación de vistas y permisos por rol médico/administrador
Actualización centralizada	Un cambio en el servidor debe reflejarse para todos de inmediato	Sí	Cada despliegue en Railway actualiza la app para todos los usuarios a la vez
Acceso completo al hardware	Debe poder usar cámara, GPS, sensores del dispositivo sin restricción	No	Solo puede usar lo que el navegador expone (a diferencia de mi app nativa de Wear OS)
Funcionamiento sin conexión	Debe seguir operando aunque no haya internet	No	MediCore requiere conexión activa; no está implementada como PWA offline

De los once criterios revisados, MediCore cumple con nueve, incluyendo todos los que definen a una aplicación web como tal (sin instalación, backend en servidor, base de datos centralizada, multiplataforma, acceso por URL, autenticación segura, protección de formularios, control de acceso por rol y actualización centralizada). Los dos criterios que no cumple —acceso completo al hardware y funcionamiento sin conexión— son, precisamente, las dos limitaciones estructurales de cualquier aplicación web frente a una nativa, y no fallas de implementación: son la razón misma por la que en el Subproducto No. 1 documenté un enfoque distinto (nativo) para el caso de uso de Wear OS, donde ese acceso al hardware sí era indispensable.

10. Lista de Cotejo

Reviso, a manera de autoevaluación, que el documento cumpla con todos los puntos solicitados para esta actividad:

ELEMENTO SOLICITADO	CUMPLE	OBS.
Portada con formato oficial UTP	Sí	
Índice / tabla de contenido	Sí	
Introducción	Sí	
Definición de qué es una aplicación web	Sí	
Ilustración del funcionamiento (diagrama)	Sí	
Características principales	Sí	
Ventajas y desventajas	Sí	
Tabla comparativa (web vs. nativa vs. híbrida)	Sí	
Tecnología utilizada	Sí	
Ejemplo de mi propia aplicación web (MediCore)	Sí	
Imágenes de mi aplicación (evidencia propia)	Sí	14 capturas reales del sistema
Tabla comparativa: requisitos vs. mi aplicación	Sí	
URL de la aplicación de ejemplo	Sí	
Correo y contraseña de acceso a la aplicación	Sí	
Conclusión	Sí	
Referencias	Sí	

11. Conclusión

Las aplicaciones web representan el enfoque de desarrollo más accesible y de menor fricción para el usuario final: no requieren instalación, funcionan igual en cualquier dispositivo con navegador, y se actualizan de forma centralizada sin intervención del usuario. A cambio, dependen de una conexión a internet y tienen acceso más limitado al hardware del dispositivo en comparación con una aplicación nativa, tal como comprobé en la tabla comparativa de la sección 9: de once criterios, MediCore cumple los nueve que definen a una app web, y solo falla en los dos que dependen del hardware y de funcionar sin conexión, que son justamente el terreno de la aplicación nativa que documenté en el Subproducto No. 1.

El desarrollo de MediCore Professional System como aplicación web fue una decisión adecuada para su caso de uso: un sistema de gestión clínica que necesita ser accedido por distintos médicos desde distintos dispositivos (computadoras de consultorio, tablets, teléfonos), sin depender de que cada uno instale y mantenga actualizada una aplicación nativa distinta para cada sistema operativo. El desplegarla en la nube, además, permite que cualquier persona con la URL y credenciales válidas pueda acceder al sistema desde cualquier parte, tal como se documenta en la siguiente sección.

Comparar en este par de subproductos el mismo tipo de análisis —definición, características, ventajas y desventajas, comparativa y tabla de cumplimiento— pero aplicado primero a un proyecto nativo (Wear OS) y ahora a uno web (MediCore) me deja mucho más claro que la elección entre nativo, web e híbrido no es una cuestión de cuál es “mejor” en abstracto, sino de qué tanto pesa cada criterio (rendimiento, acceso al hardware, costo, mantenimiento) en el proyecto específico que se está construyendo.

12. URL de la Aplicación

Como evidencia del ejemplo presentado (MediCore Professional System), se listan sus datos de acceso:

- URL de acceso: <https://practicaequipogithub-production.up.railway.app/home.php>
- Repositorio de código fuente:
<https://github.com/AlexAP1115web/PracticaEquipoGitHub>

PASOS: en la página principal, dar clic en el botón “Acceso Médico” para abrir el modal de inicio de sesión, e ingresar las siguientes credenciales:

- Correo: haydee.gomez@utpuebla.edu.mx
- Contraseña: MediCore2026!

Referencias

- HubSpot. (2026). Qué es Frontend y Backend: guía con diferencias, ejemplos y herramientas. <https://blog.hubspot.es/website/frontend-y-backend>
- Platzi. (2026). Qué es Frontend y Backend: principal diferencia y ejemplos. <https://platzi.com/blog/que-es-frontend-y-backend/>
- UNIR. (2026). Back-end vs. front-end: ¿cuáles son sus diferencias?. <https://unirfp.unir.net/revista/ingenieria-y-tecnologia/back-end-front-end/>
- PHP Documentation Team. (2026). PHP Manual. <https://www.php.net/docs.php>
- MySQL Documentation Team. (2026). MySQL Reference Manual. <https://dev.mysql.com/doc/>
- Railway. (2026). Railway Documentation. <https://docs.railway.com>
- Mozilla Developer Network. (2026). Progressive web apps (PWAs). https://developer.mozilla.org/es/docs/Web/Progressive_web_apps
- OWASP Foundation. (2026). OWASP Top 10. <https://owasp.org/www-project-top-ten/>